WORKSHEET MAHASISWA

Modul 3 Special Challenge : Advanced CRUD dan Data Integrity

Mata Kuliah: Proyek 3 - Proyek Pengembangan Perangkat Lunak Berbasis Web · D3 Teknik Informatika

Nama	Hafiez Novrizal Putra
NIM	251511011
Kelas	2A-D3
Tanggal Mulai	28/9/26
Tanggal Pengumpulan	02/10/2026
URL Repository GitHub	https://github.com/hafieznp611/Project-3-Semester-3
Branch/Tag Modul	modul3-special-challenge
Commit Baseline	
Petunjuk Penggunaan	
Isi worksheet ini bersamaan dengan pengerjaan challenge. Tuliskan bukti singkat, commit hash, tangkapan layar yang relevan, dan penjelasan teknis. Worksheet dibuat ringkas, tetapi setiap checkpoint harus dapat ditelusuri melalui repository dan bukti pengujian.	

A. Baseline dan Rencana Checkpoint

Pastikan CRUD dasar kegiatan sudah berjalan sebelum memulai Special Challenge. Jika baseline belum stabil, perbaiki terlebih dahulu sebelum menambah relasi, business rule, lifecycle, atau transaction.

Checkpoint	Isi Minimum	Commit/Tag	Status
Baseline	CRUD kegiatan dasar berjalan	Worksheet 3 Special Challenge Final	☐
Task 1	Relationship, foreign key, unique, delete policy	Worksheet 3 Special Challenge Final	☐
Task 2	Business rule, state transition, search, filter, sort, pagination	Worksheet 3 Special Challenge Final	☐
Task 3	Soft delete, restore, eager loading, SonarQube review	Worksheet 3 Special Challenge Final	☐
Add-on	Poster storage dan refactoring	Worksheet 3 Special Challenge Final	☐

B. Peta Relasi dan Integritas Data

Lengkapi tabel berikut untuk memastikan model data, relationship, dan kebijakan integritas dapat dijelaskan sebelum implementasi.

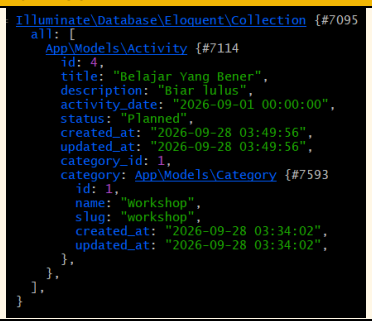
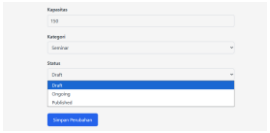
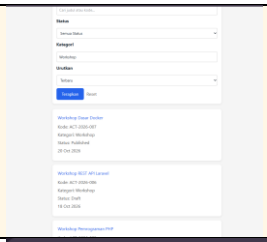
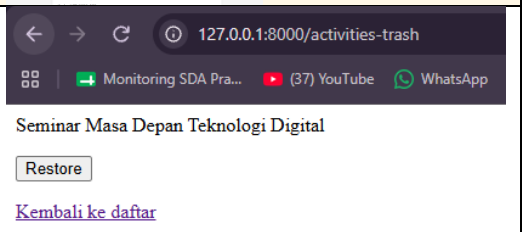
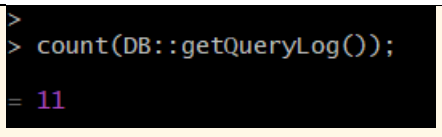
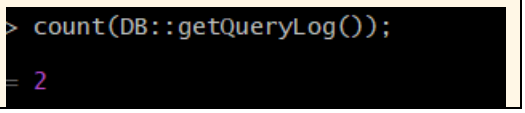
Relasi	Foreign Key / Constraint	Kebijakan	Bukti
Category hasMany Activity	Tidak ada fk langsung di tabel categories, dijaga dari sisi activities.	Kategori yang masih dipakai tidak boleh dihapus.	
Activity belongsTo Category	Activities.category_id -> categories.id, via $table->foreighId(category_id)->constrained->restrictOnDelete()	Setiap activity wajib memiliki kategori valid.	
Activity hasMany Registration	Tidak ada fk langsung di tabel activities dijaga dari sisi registrations	Pendaftaran hanya untuk activity yang memenuhi rule.	
Registration belongsTo Activity	registrations.activity_id → activities.id, via $table->foreignId('activity_id')->constrained(); plus unique(['activity_id','email']) untuk IC-03.	Tidak boleh menunjuk activity yang tidak tersedia.	

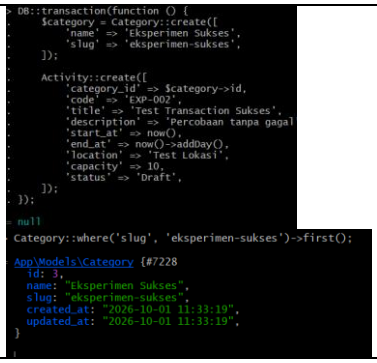
Catatan singkat strategi migrasi jika tabel activities telah berisi data sebelum category_id dibuat wajib:

Jika category_id langsung dibuat NOT NULL/constrained padahal tabel activities sudah berisi data lama tanpa category_id, migration akan gagal dijalankan karena database tidak tahu nilai apa yang harus diisi untuk baris yang sudah ada.
Strategi yang dapat dipakai yaitu buat kolom category_id nullable dahulu, isi data lama secara manual/seeder, baru ubah menjadi NOT NULL di migration terpisah atau jika data lama hanya data uji, lakukan migratefresh dan seed ulang dengan category_id sudah terisi sejak awal.
Saya memilih migrate:fresh dijalankan, lalu CategorySeeder dan ActivitySeeder diperbarui agar setiap activity langsung memiliki category_id yang valid sejak data dibuat.

C. Ringkasan Eksperimen Konsep

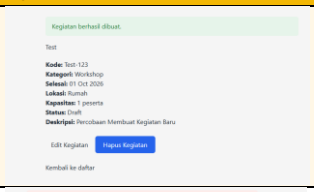
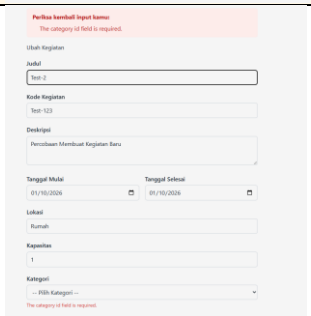
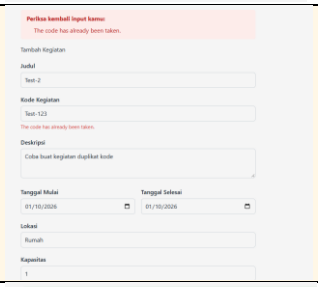
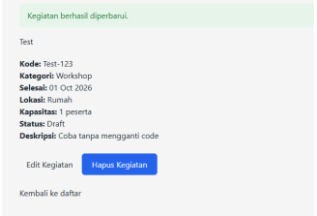
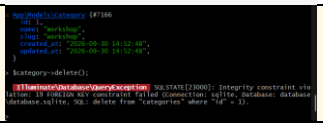
Gunakan bagian ini untuk mencatat hasil eksperimen penting secara singkat. Bukti detail dapat berupa nama file tangkapan layar atau tautan commit.

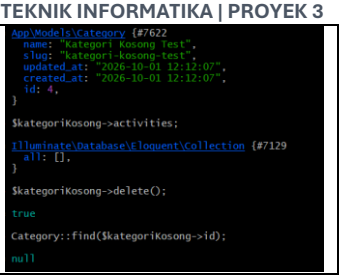
Eksperimen	Prediksi Singkat	Hasil Aktual	Bukti/Commit
Relationship dan referential integrity	Bagaimana data saling memiliki hubungan dan tetap menjaga data tersebut agar tetap saling terhubung dan akurat satu sama lain.	Setiap activity dapat memiliki karegory yang sudah terdaftarkan.	
Advanced validation dan business rule	Memastikan bahwa data yang dimasukan akurat, aman, dan sesuai dengan kebutuhan sistem.	Validasi format category_id harus ada di database, status harus salah satu dari Planned/Ongoing/Published ditangani Form Request. Business rule yang bergantung pada state sebelumnya (transisi status tidak boleh mundur	
Search, filter, sort, pagination	Digunakan untuk mengelola, mengurutkan, dan menampilkan data agar lebih mudah untuk dibaca.	Filter dapat digunakan baik di filter berdasarkan kategori maupun tanggal dimaan Ketika di filter berdasarkan kategori maka hanya kategori yang dipilih yang akan muncul	
Soft delete dan restore	Ketika data dihapus sebenarnya di data base,data tersebut tidak benar-benar dihapus, melaikan di ubah statusnya, agar tidak bisa diakses selain di data base.	Data yang dihapus tidak hilang dan dapat dikembalikan.	
N+1 dan eager loading		Ketika tanpa eager loading maka jumlah query yang muncul adalah 11 sedangkan Ketika dengan eager laoding jumlah querynya 2	 

Transaction dan rollback terkontrol	Category & Activity tidak tersimpan (rollback)	Category::where('slug','eksperimen-transaction')->first() = null	
	Category & Activity tersimpan normal	Category::where('slug','eksperimen-sukses')->first() = data lengkap	

D. Task 1 - Relational CRUD dan Integrity

Target: kategori terhubung dengan kegiatan; foreign key, unique constraint, dan kebijakan delete dapat dibuktikan.

Skenario Uji	Ekspektasi	Status	Bukti
Create dengan kategori valid	Data tersimpan dan relasi benar.	☐	
Create dengan category_id tidak valid	Request atau database menolak data.	☐	
Create kode duplikat	Request ditolak; unique tetap terjaga.	☐	
Update tanpa mengganti kode	Tidak dianggap duplikat terhadap dirinya sendiri.	☐	
Hapus kategori yang masih dipakai	Ditolak; data kegiatan tetap utuh.	☐	

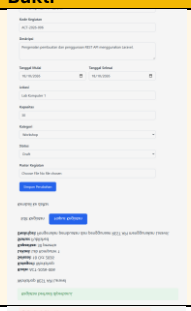
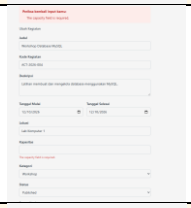
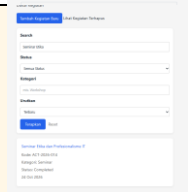
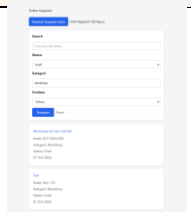
Hapus kategori kosong	Berhasil sesuai kebijakan aplikasi.	☐	

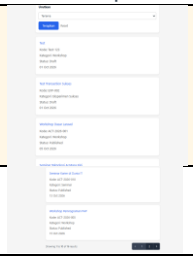
Penjelasan ringkas: mengapa relationship Eloquent tidak menggantikan foreign key database?

Relationship Eloquent (hasMany/belongsTo) membantu aplikasi membaca hubungan data misal $activity->category, tapi tidak mencegah data tidak konsisten masuk ke database, misalnya lewat query manual, seed data, atau proses lain di luar Eloquent. Foreign key di database adalah penjaga terakhir yang menolak secara paksa setiap percobaan insert/delete yang melanggar integritas referensial, apa pun jalur yang dipakai untuk mengaksesnya.

E. Task 2 - Business Rule dan Query Experience

Target: state transition dikendalikan secara eksplisit dan daftar kegiatan dapat dicari, difilter, diurutkan, serta dipaginasi.

Rule/Query	Lapisan Implementasi	Hasil Uji	Bukti
Draft lengkap ke Published	Service / method khusus	Berhasil, status berubah jadi Published	
Draft tidak lengkap ke Published	Business rule guard	Ditolak, pesan menyebutkan field yang kurang	
Published ke Completed	Service / method khusus	Berhasil, status berubah jadi Completed	
Completed ke Draft	Transisi ilegal ditolak	Ditolak, pesan "Transisi status Completed ke Draft tidak diizinkan."	
Search code atau title	Query database	Hasil sesuai keyword, hanya tampilkan judul/kode yang cocok	
Filter kategori dan status	Query database	Hasil sesuai kombinasi filter	

Sort tanggal terbaru/terlama	Query database	Urutan berubah sesuai pilihan	
Pagination dengan query aktif	withQueryString()	Filter tetap aktif saat pindah halaman	

Keputusan teknis: bagian mana yang Anda tempatkan pada Form Request dan bagian mana yang Anda tempatkan pada service?

Form Request (StoreActivityRequest/UpdateActivityRequest) menangani validasi bentuk input format tanggal (date, after_or_equal), tipe data (integer, string), panjang field, keberadaan category_id di database (exists), dan keunikan code (unique). Field status sengaja tidak divalidasi di sini karena perubahannya tidak datang dari form bebas.

ActivityService menangani aturan yang bergantung pada state transisi status (ensureValidTransition) membutuhkan pendefisinisian awal misal dari Draft hanya bisa draft lagi atau lanjut ke Pubished, status activity sebelumnya (current state), bukan sekadar format data baru. Kelengkapan field sebelum publish (ensureReadyToStart) juga ditempatkan di service karena aturan ini hanya berlaku pada transisi Draft ke Published, bukan aturan input yang berlaku kalau ditaruh di Form Request, aturan ini akan ikut menghalangi proses create/update biasa yang semestinya boleh menyimpan data draft yang belum lengkap.

Publish dan Complete dipicuu lewat route terpisah (PATCH /activities/{id}/publish dan /complete) yang langsung memanggil ActivityService::update() dengan payload minimal {status: ...}, tanpa melalui UpdateActivityRequest

F. Task 3 - Data Lifecycle dan Query Quality

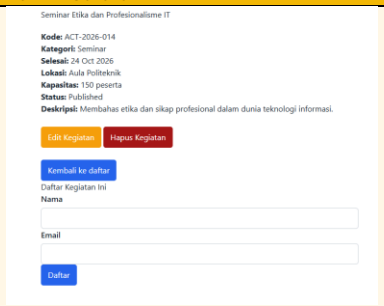
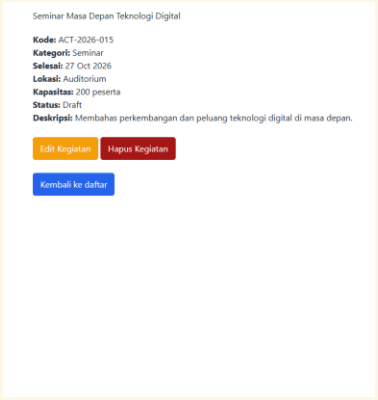
Target: soft delete dan restore berjalan, index tidak menampilkan data terhapus, dan daftar activity tidak menghasilkan pola N+1 yang tidak diperlukan.

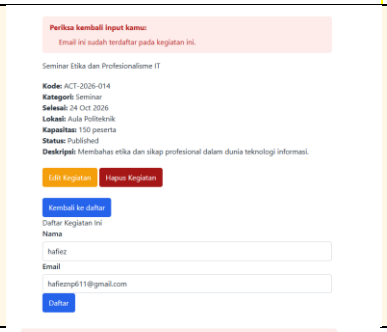
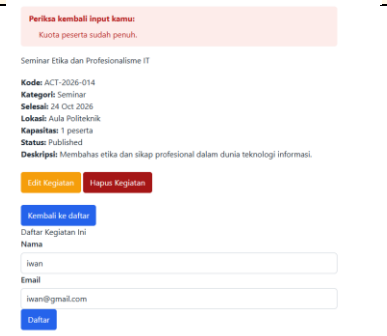
Acceptance Criterion	Status	Bukti	Catatan Teknis
AC-10 Soft delete	☐	Data hilang dari halaman index setelah delete	Ketika di delete activities tetap ada hanya di ubah statusnya menjadi deleted_at.
AC-11 Restore	☐	Halaman trash menampilkan data terhapus; setelah klik restore, data kembali muncul di index normal	Ketika membuka halaman trash maka route restore menggunakan withtrashed() agar route dapat menemukan deleted_at
AC-12 Tidak ada pola N+1 pada index	☐	Query log: lazy loading = 11 query, eager loading with('category') = 2 query	with('category') dipasang pada query index() dan trash()
AC-13 Struktur tanggung jawab tetap jelas	☐	Review kode: controller tetap sebagai orchestrator	destroy()/restore() tetap ringkas karena logic soft delete ditangani trait Eloquent, bukan ditulis manual di controller, tidak ada query kategori manual di controller karena sudah lewat eager loading di query builder
AC-14 Satu refaktorisasi berbasis static analysis	☐		

Pendekatan	Jumlah Query	Kesimpulan
Lazy loading pada loop	11	Ketika tidak menggunakan eager loading maka query akan terjadi 1 kali + dengan mengambil 10 activity 1 per 1.
Eager loading with(category)	2	Sedangkan Ketika mengguankan eager loading 1 query mengambil activity, ditambah 1 query akan mengambil semua kategori terkait sekaligus.

G. Independent Challenge - Atomic Registration

Target: pendaftaran peserta menggunakan transaction sehingga Registration dan registered_count konsisten.

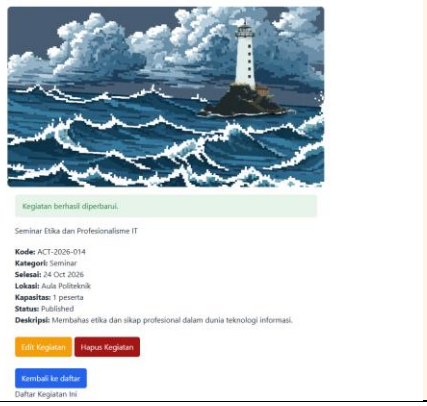
Aturan	Status	Bukti / Catatan
Pendaftaran hanya untuk activity published.	☐	 

		Ketika status selain published maka menu pendaftaran tidak akan tampil. @if ($activity->status === 'Published')
Pendaftaran ditolak jika start_at sudah lewat.	☐	
Email yang sama tidak boleh mendaftar dua kali pada activity yang sama.	☐	
Jumlah pendaftar tidak boleh melebihi capacity.	☐	
Registration dan registered_count berada dalam satu transaction.	☐	 Dimana Ketika transaction terjadi otomatis registered count bertambah.
Rollback terkontrol membuktikan tidak ada data parsial.	☐	

Penjelasan ringkas: mengapa validation saja tidak menggantikan transaction?

Validation hanya memastikan data yang masuk sudah benar sebelum proses dimulai, tapi tidak bisa menjamin apa yang terjadi ketika proses tersebut terdiri dari beberapa operasi database berurutan.

H. Add-On Poster Storage dan Refactoring

Pemeriksaan	Status	Bukti / Keputusan
Poster valid dapat diunggah dan ditampilkan.	☐	
File tidak sesuai ditolak dengan pesan yang jelas.	☐	Hanya dapat menginputkan file jpg, png dan image lainnya <input type="file" id="poster" name="poster" accept="image/*">
Penggantian poster tidak meninggalkan file lama tanpa alasan.	☐	Poster yang baru akan mereplace poster yang lama ditangani oleh update() karena di update() akan menyimpan path lama kemudian menyimpan file baru lalu meupdate database kemudian hapus path lama.
Soft delete tidak merusak kemungkinan restore poster.	☐	Poster tidak ikut terhapus ketika soft delete
Minimal satu refaktorisasi dijelaskan.	☐	Tadinya atnggung jawab pengecekan untuk replace poster ada didalam method update() di activity service , jadi saya pindahkan agar tanggung jawab untuk me replace poster di buat sendiri, sehingga pembagian tanggung jawab jelas.

I. Quality Check dan Static Analysis

Temuan SonarQube	Kategori Keputusan	Perubahan	Bukti Uji Ulang
Temuan yang paling saya temukan adalah missing line at the end of file	☐ Relevan ☐ Perlu konteks ☐ Tidak relevan	Tidak ada perubahan yang cukup berrpengaruh hanya menambah line di akhir saja	
Poster harus ada altnya	☐ Relevan ☐ Perlu konteks ☐ Tidak relevan	Menambahkan atribut alt="Poster {{ $activity->title }}" pada tag <img> yang menampilkan poster kegiatan	
	☐ Relevan ☐ Perlu konteks ☐ Tidak relevan		

Refaktorisasi yang dilakukan dan alasan teknisnya:

J. Self-Assessment dan Refleksi Akhir

Pernyataan	Skor 1-5
Saya dapat menjelaskan perbedaan Eloquent relationship dan foreign key.	3
Saya dapat membedakan validation rule, database constraint, dan business rule.	3
Saya dapat menjelaskan mengapa transisi status perlu dikendalikan.	3
Saya dapat membangun search, filter, sort, dan pagination sebagai query database.	2
Saya dapat menunjukkan masalah N+1 dan memperbaikinya dengan eager loading.	2
Saya dapat menjelaskan transaction dan rollback.	2
Saya dapat membaca hasil SonarQube dan menentukan temuan relevan.	1

Konsep apa yang paling mengubah cara Anda melihat CRUD setelah challenge ini?

Ternyata CRUD bukan Cuma soal create,input, delete, update dibaliknya ada banyak sekali rule yang bekerja seperti tidak boleh sama atau unique, ada yang harus diisi dan tidak boleh dikosongkan, ada yang harus integer, ada yang harus karakter dll.

Business rule mana yang paling sulit ditempatkan dan mengapa?

Business rule soft delete dan harus bisa dipulihkan, setiap kegiatan harus memilih kategory yang ada di database, karena di baliknya ada beberapa data yang harus saling berhubungan (memiliki realtionship).

Apa bukti yang membuat Anda memahami transaction atau rollback?

Transaction ad ajika ada lebeih dari satu perubahan database yang harus dianggap sebagai sebuah kesatuan, dimana Ketika menambah registration otomatis ada penambahan di database yaitu orang yang melakukan registration dan juga ada penambahan kapasitas dari aktifitas yang akan berlangsung walaupun penambahn bukan langsung pada aktifitasnya melainkan akan ada penghitungan tersendiri untuk membandingkan apakah peserta sudah mencapai kapasitas atau belum yaitu melalui registration_count yang akan di bandingkan dengan kapasitas aktifitas..

Jika aplikasi ini dilanjutkan ke authentication dan authorization, bagian mana yang kemungkinan berubah?

Di bagian CRUD activity dimana kitab isa mengubah keterangan kegiatan seperti jadwal, tempat, status dll sebaiknya Ketika sudah mulai di terapkan authorization yang bisa mengubah hanya admin atau yang memiliki wewenang saja misal penyedia activitynya.

K. Checklist Pengumpulan

Item	Status
Worksheet PDF telah diisi ringkas dan bukti dapat dibaca.	☐
URL repository GitHub dan commit/tag setiap checkpoint tersedia.	☐
Aplikasi dapat dijalankan dan migration berjalan tanpa error.	☐
Acceptance criteria utama telah diuji.	☐
Kode eksperimen sementara sudah dihapus dari branch final.	☐
Mahasiswa siap menjelaskan satu perubahan business rule saat demonstrasi.	☐